CSE 351 - Programming Languages

Term Project Report: Dead Code Elimination

Ece Düzgeç
20220702045

ABSTRACT

This project implements a dead code elimination algorithm for an intermediate language using Lex and Yacc. The implementation consists of a lexical analyzer that tokenizes input statements, a parser that builds internal data structures, and an optimization engine that performs backward liveness analysis to eliminate unnecessary computations, where the system was tested with five input files containing 4 to 14 statements, successfully eliminating dead code while preserving correct values for all live variables. The implementation demonstrates practical application of compiler construction tools and dataflow analysis techniques.

Contents

1. INTRODUCTION

Dead code elimination is a compiler optimization that removes assignment statements whose computed values are never used. This project implements this optimization for a simple intermediate language where each statement has at most two operands and programs end with a list of live variables.

The implementation uses Lex (Flex) to perform lexical analysis and Yacc (Bison) to parse the grammar and build data structures. The optimization algorithm then processes statements in reverse order, maintaining a set of live variables and keeping only statements that contribute to computing final live values.

The project structure consists of:

- **termproject.l:** Lexical analyzer specification defining token patterns
- **termproject.y:** Parser specification with grammar rules and optimization logic
- **Makefile:** Build automation
- **Input files:** Five test cases demonstrating different scenarios

2. DESIGN AND IMPLEMENTATION

2.1 Lexical Analyzer Implementation (termproject.l)

The lexical analyzer recognizes tokens through pattern matching rules. Each operator and delimiter has a simple recognition rule:

"=" : ASSIGN "+" :PLUS "-" : MINUS "*" : MULT "/" : DIV "^" :XOR

";" :SEMICOLON "{" : LBRACE "}" : RBRACE "," :COMMA

For identifiers and numbers, regex patterns are used:

Identifier Pattern: [a-zA-Z][a-zA-Z0-9_]*

- Matches any sequence starting with a letter, followed by letters, digits, or underscores
- Action: yylval.str = strdup(yytext); copies the matched text to pass to the parser
- The strdup() function allocates new memory because yytext gets overwritten

Number Pattern: [0-9]+

- Matches one or more consecutive digits
- Action: yylval.num = atoi(yytext); converts the string to an integer
- The atoi() function converts text like "123" to integer value 123

Whitespace Handling: [\t\n]+

- Matches spaces, tabs, and newlines
- Empty action means ignore this input

2.2 Parser Data Structures (termproject.y)

The parser which analyzes the code uses two main data structures to represent the program:

Statement Structure:

typedef struct {

char dest[MAX_VAR_NAME];

char op1[MAX_VAR_NAME];

char op2[MAX_VAR_NAME];

char operator; int is_const1;

int is_const2; int has_op2;

} Statement;

- This design allows both variables and constants to use the same character array field, simplifying the code. When printing output, we just output the string directly regardless of whether it represents a variable name or a numeric value.

Example statement representations:

a = 5; becomes: dest="a", op1="5", op2="", operator=0, is_const1=1, is_const2=0, has_op2=0

x = y + 3; becomes: dest="x", op1="y", op2="3", operator='+', is_const1=0, is_const2=1, has_op2=1

LiveSet Structure:

typedef struct {

char vars[MAX_STATEMENTS][MAX_VAR_NAME];

int count; } LiveSet;

- This structure maintains the set of variables that are currently live during backward analysis.

2.3 Grammar Rules

The parser must handle many statement forms. Here are key examples showing the pattern:

Simple assignment (variable = variable):

IDENTIFIER ASSIGN IDENTIFIER SEMICOLON {

add_statement($1, $3, NULL, 0, 0, 0, 0); }

Assignment with number (variable = number):

IDENTIFIER ASSIGN NUMBER SEMICOLON {

char num_str[20]; sprintf(num_str, "%d", $3);

add_statement($1, num_str, NULL, 0, 1, 0, 0); }

Here we convert the NUMBER token to a string before storing it.

Binary operation (variable = variable + variable):

IDENTIFIER ASSIGN IDENTIFIER PLUS IDENTIFIER SEMICOLON {

add_statement($1, $3, $5, '+', 0, 0, 1); }

The parser has 24 statement rules covering all combinations: 2 simple assignment forms (var=var, var=num) plus 5 operators * 4 operand combinations (var op var, var op num, num op var, num op num) = 20 rules.

2.4 Dead Code Elimination Algorithm

The optimization is performed in process_dead_code_elimination():

Step 1: Initialize live set with final live variables

current_live_set.count = 0;

for (int i = 0; i < live_var_count; i++) {

add_to_live_set(live_vars[i]); }

Step 2: Process statements in reverse order

for (int i = stmt_count - 1; i >= 0; i--) {

if (is_live(statements[i].dest)) {

// Keep this statement

output_statements[output_count] = statements[i];

```
output_count++;
 // Add variable operands to live set
 if (!statements[i].is_const1) {
add_to_live_set(statements[i].op1);        }
 if (statements[i].has_op2 && !statements[i].is_const2) {
add_to_live_set(statements[i].op2);        }
// Remove destination from live set
 remove_from_live_set(statements[i].dest);     } }
```

The algorithm works backwards because we need to know which variables are needed later before we can determine if a statement is dead. When we keep a statement, we add its variable operands to the live set as constants don't need to be computed and remove the destination since it becomes live only at this statement.

3. ALGORITHM EXPLANATION

To understand how the algorithm works, let's trace through input1.txt step by step.

Input:
a=2+2; b=2^9; c=d^3; e=5; f=3*4; g=6/2; h=m; p=0; j=j+p; r=e*p; s=a; { r, s }

Initialization: Statements array contains 11 statements (indices 0-10). Live set: { r, s }

Processing (statements 10 down to 0):

Statement 10: s=a;

- Check: Is 's' live? →Yes it is in { r, s }
- Action: Keep statement, add 'a' to live set, remove 's'
- New live set: { r, a }

Statement 9: r=e*p;

- Check: Is 'r' live? →Yes
- Action: Keep statement, add 'e' and 'p' to live set, remove 'r'
- New live set: { a, e, p }

Statement 8: j=j+p;

- Check: Is 'j' live? →No
- Action: Skip this statement (Dead Code)

Statement 7: p=0;

- Check: Is 'p' live? →Yes
- Action: Keep statement, operand is constant (don't add), remove 'p'
- New live set: { a, e }

Statements 6, 5, 4, 2, 1: h=m;, g=6/2;, f=3*4;, c=d^3;, b=2^9; are all Dead Code since destinations are not live

Statement 3: e=5;

- Check: Is 'e' live? →Yes
- New live set: { a }

Statement 0: a=2+2;

- Check: Is 'a' live? →Yes
- New live set: { } (empty)

Output array (indices 0-4):

- Index 0: s=a;
- Index 1: r=e*p;
- Index 2: p=0;
- Index 3: e=5;
- Index 4: a=2+2;

Printed output (reverse order):

a=2+2; e=5; p=0; r=e*p; s=a;

Six statements eliminated: b=2^9; c=d^3; f=3*4; g=6/2; h=m; j=j+p; The kept statements form two dependency chains: s ←a and r ←e,p.

4. TESTS AND RESULTS

4.1 Test Case 1: input1.txt

Output: 5 statements kept (from 11 input statements)

a=2+2; e=5; p=0; r=e*p; s=a;

Analysis: Six statements eliminated because their destinations are never used. The kept statements form two dependency chains: s ← a and r ← e,p.

4.2 Test Case 2: input2.txt

Output: 2 statements kept (from 3 input statements)

b=z+y; x=2*b;

Analysis: Statement a=b; eliminated because 'a' is never used. Statement b=z+y; kept because even though 'b' is not in the final live set, it is needed to compute 'x'.

4.3 Test Case 3: input3.txt

Output: 3 statements kept (all statements)

x=5; y=x+3; z=y*2;

Analysis: No elimination because all three variables are in the live set. Each statement computes a live variable, so all are necessary.

4.4 Test Case 4: input4.txt

Output: 10 statements kept (from 14 input statements)

a=1+1; b=2+2; c=a+b; d=10+10; e=20+20; f=c+5; g=d+e; i=f*2; j=g*3; m=i+j;

Analysis: Four statements eliminated (h, k, n, o). The dependency chain for 'm' requires: m←i,j → i←f, j←g → f ←c, g ← d, e → c ← a,b. Variables h, k, n, o are computed but never contribute to 'm'.

4.5 Test Case 5: input5.txt

Output: 5 statements kept (from 11 input statements)

a=5+5; b=10+10; c=15+15; d=20+20; e=25+25;

Analysis: Six statements eliminated (f, g, h, i, j, k). Only the five basic variables are live at the end. All derived computations are unnecessary because their results are not examined.

4.6 Results Summary

Test	Input	Output	Eliminated	Key Feature
1	11	5	6	Multiple independent dead
2	3	2	1	Intermediate variable needed
3	3	3	0	All variables live
4	14	10	4	Complex dependency chain
5	11	5	6	Long chain eliminated
Total	42	25	17	

Across all tests, 17 out of 42 statements were eliminated while maintaining correct values for all live variables.

5. CONCLUSION

This project successfully implemented dead code elimination for an intermediate language. The implementation correctly:

- Parses all valid statement forms using Lex and Yacc
- Performs backward liveness analysis to identify necessary statements
- Eliminates unnecessary computations while preserving semantics
- Handles various scenarios from zero elimination to substantial code reduction

5.1 Key Analysis

- By using constants as strings, this simplifies the Statement structure by allowing a single field type for both variables and constants. The tradeoff is slightly more memory usage, but the code clarity benefit is worth it for this application.

- For this project, fixed arrays simplify memory management. A production compiler would use dynamic allocation to handle programs of any size.
- By using different grammar rules, each combination of operand types (variable vs. constant) requires separate handling to correctly set the is_const1 and is_const2 flags. This ensures the optimizer knows which operands need to be marked live.
- With Compiler Construction we determine how pattern matching and grammar rules generate working code analyzers.
- With Implementation of dataflow analysis where information flows from program end to beginning. Understanding how to maintain live set state and make decisions based on that state.
- By designing algorithms that work backward through data structures, efficiently manage sets, and make correctness-preserving transformations.

6. REFERENCES

1. Yeditepe University. (2025). CSE351.3- Programming Languages] [Course Material, Accessed via YULearn]. https://yulearn.yeditepe.edu.tr/course/view.php?id=42351

2. Robert W. Sebesta, “Concepts of Programming Languages”

3. T.W. Pratt and M.V. Zelkowitz, Programming Languages, Design and Implementation

4. GeeksforGeeks. "Introduction of Lexical Analysis." https://www.geeksforgeeks.org/introduction-of-lexical-analysis/

5.GeeksforGeeks. "Introduction to Syntax Analysis in Compiler Design." https://www.geeksforgeeks.org/introduction-to-syntax-analysis-in-compiler-design/

6.Stack Overflow. "How does Lex/Flex work internally?" https://stackoverflow.com/questions/tagged/flex+lex

7. Reddit r/Compilers. "Dead Code Elimination Techniques Discussion." https://www.reddit.com/r/Compilers/